Iteración 1: pruebas y SUT

Apartados 2 y 3 de la Entrega 2

Alcance y estado

La propuesta de IT1 comprende HU-01 (registro), HU-02 (inicio de sesión), HU-05 (alta de lugares por coordenadas), HU-07 (consulta de lugares) y HU-14 (cálculo de rutas). Su distribución definitiva en la planificación global se acordará con el equipo.

Se han programado 25 pruebas para los escenarios identificados en la Entrega 1. El SUT mínimo define constructores, métodos, tipos y dependencias para poder ejecutarlas. Los métodos nuevos lanzan un error de funcionalidad pendiente. Esta ejecución documenta la fase roja de ATDD; la implementación funcional y la fase verde corresponden al desarrollo posterior.

Historia	Pruebas	Escenarios identificados
HU-01	6	Un registro válido y cinco escenarios inválidos.
HU-02	5	Un acceso válido y cuatro escenarios inválidos.
HU-05	5	Un alta válida y cuatro escenarios inválidos.
HU-07	4	Dos consultas válidas y dos escenarios inválidos.
HU-14	5	Un trayecto válido y cuatro escenarios inválidos.
Total	25	Resultado de ejecución: 0 pasan, 25 fallan.

Ejecución reproducible y evidencia

Desde la raíz del repositorio, con Node.js compatible con los módulos y la transformación TypeScript usados por el spike, ejecutar en PowerShell:

```
$pruebas = Get-ChildItem tests -Filter '*.aceptacion.test.ts' | Select-Object -ExpandProperty FullName
node --experimental-transform-types --env-file-if-exists=entrega1/02_Spike/.env --test-reporter=spec --test $pruebas
```

La evidencia conserva versión de Node, fecha UTC, commit de código y salida completa. Se verificó que las 25 pruebas se ejecutan, que el proceso devuelve código 1 y que los fallos proceden de los métodos pendientes, sin errores de carga o sintaxis. Ver evidencia de ejecución.

Dependencias. Las pruebas de datos usan SQLite real en memoria y preparan sus propios datos. El caso válido de ruta recibe el adaptador real de ORS y necesitará `ORS_API_KEY` al implementar el servicio. En la fase roja todavía no llega a consultar ORS.

Indisponibilidad. Los casos de conexión preparan un servidor HTTP local que corta la conexión. En E-14-I3 se redirige la dependencia HTTP del adaptador a ese servidor; se trata de una comprobación de integración con el fallo controlado de un sustituto del proveedor. Esta clasificación debe conservarse en la documentación de pruebas.

Código completo. `HU-01: registro`; `HU-02: sesion`; `HU-05: alta de lugar`; `HU-07: consulta de lugares`; `HU-14: rutas`.

Ejemplos de pruebas de aceptación

Los ejemplos siguientes reproducen dos pruebas de la HU-01. Antes de cada prueba se crea una base SQLite vacía con la tabla `cuentas`. `datosDeAna` contiene nombre Ana, correo `ana@example.com` y contraseña `RutaSegura2026`. Los imports, el esquema y la preparación completa están en el archivo enlazado.

PA-01-V: registro válido

```
// PA-01-V (E-01-V).
prueba("registrar_correoNuevo_creaCuentaYDevuelveIdentificador", () => {
  // Dado un correo sin registrar.
  const servicio = new ServicioCuentas(baseDeDatos);

  // Cuando Ana se registra.
  const resultado = servicio.registrar(datosDeAna);

  // Entonces se guarda una cuenta y se devuelve su identificador.
  const cuentas = baseDeDatos.prepare("SELECT identificador, correo FROM cuentas").all();
  comprobarIgualdad(cuentas.length, 1);
  comprobarIgualdad(cuentas[0]?.correo, datosDeAna.correo);
  comprobarIgualdad(resultado, { identificador: cuentas[0]?.identificador });
});
```

Se comprueban la creación de una única cuenta, su correo y que la respuesta contiene únicamente el identificador de la cuenta guardada.

PA-01-I: correo duplicado

```
// PA-01-I (E-01-I1).
prueba("registrar_correoDuplicado_rechazaAltaSinCrearCuenta", () => {
  // Dado que Ana ya tiene una cuenta.
  baseDeDatos.prepare("INSERT INTO cuentas VALUES (?, ?, ?, ?)").run(
    "cuenta-ana",
    datosDeAna.nombre,
    datosDeAna.correo,
    "hash-de-prueba",
  );
  const servicio = new ServicioCuentas(baseDeDatos);

  // Cuando repite el alta, se rechaza por correo duplicado.
  comprobarError(() => servicio.registrar(datosDeAna), /correo duplicado/i);

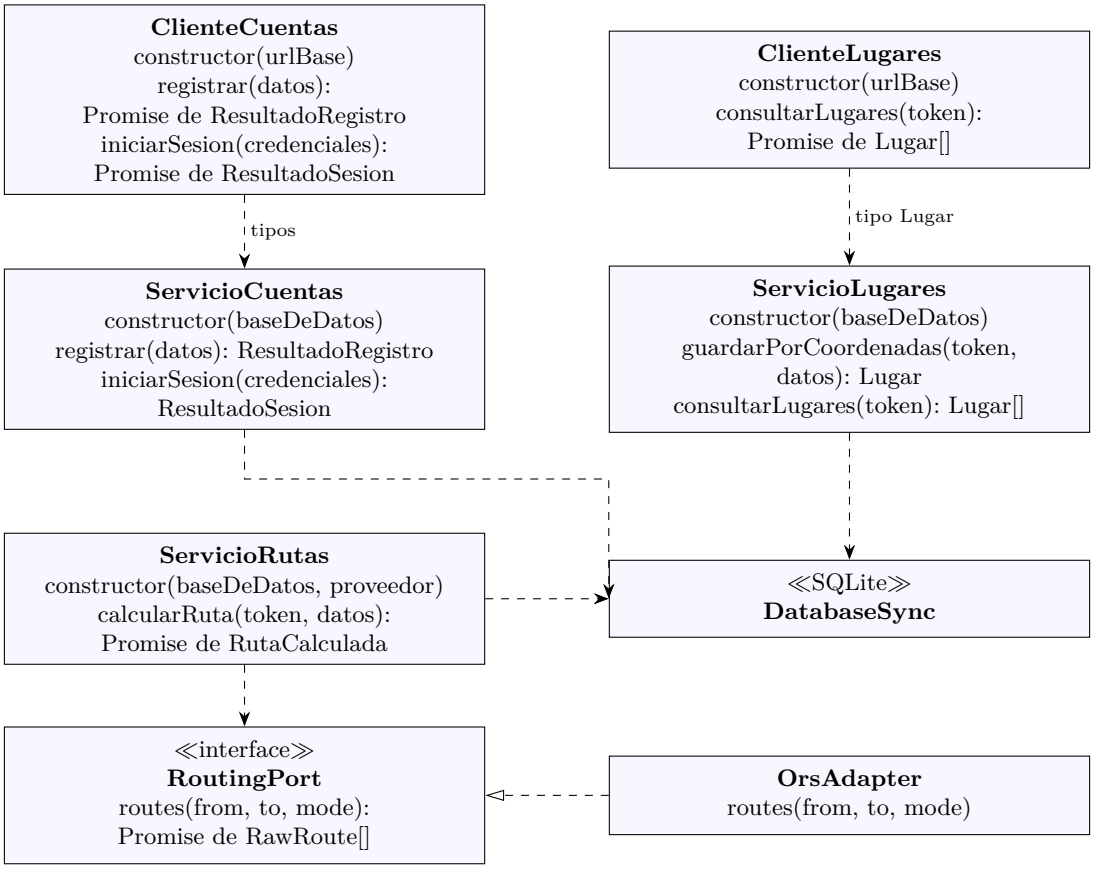
  // Entonces sigue existiendo solo la cuenta original.
  const cuentas = baseDeDatos.prepare("SELECT identificador FROM cuentas").all();
  comprobarIgualdad(cuentas.length, 1);
  comprobarIgualdad(cuentas[0]?.identificador, "cuenta-ana");
});
```

Se prepara una cuenta existente, se intenta repetir el alta y se exige un error de correo duplicado. La comprobación final conserva una sola cuenta con su identificador original. Cada prueba prepara su estado; la prueba inválida utiliza una inserción directa para evitar depender de que funcione el registro.

Los nombres siguen `metodoBajoPruebas_escenario_comportamientoEsperado`. Los comentarios separan Dado, Cuando y Entonces. En la fase actual ambas pruebas fallan al alcanzar `registrar`, que lanza `Registro sin implementar`.

UML del sistema bajo prueba

Las clases se corresponden con `ServicioCuentas`, `ServicioLugares`, `ServicioRutas`, `ClienteCuentas` y `ClienteLugares`. Las flechas discontinuas representan dependencias; la punta triangular vacía indica implementación de la interfaz.



Datos y resultados. `DatosRegistro` agrupa nombre, correo y contraseña. `ResultadoRegistro` devuelve el identificador. `Credenciales` agrupa correo y contraseña. `ResultadoSesion` devuelve token y cuenta pública. `Lugar` contiene identificador, nombre, latitud y longitud. `SolicitudRuta` contiene los identificadores de origen y destino y el modo. `RutaCalculada` contiene geometría, distancia en metros y duración en segundos. Estos nombres abrevian los tipos definidos en línea en el código.

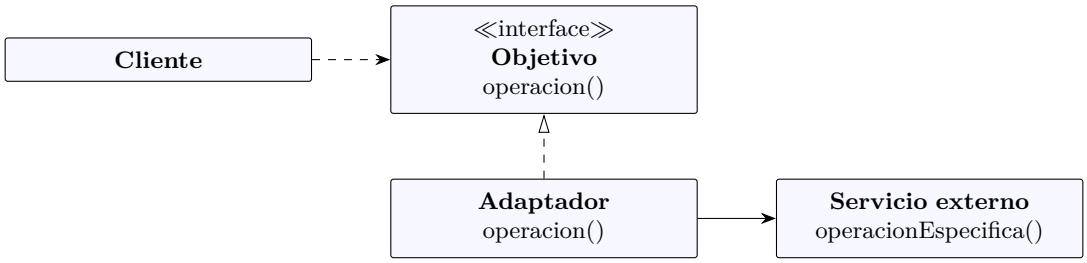
Los constructores reciben las dependencias que preparan las pruebas. Las dependencias de los clientes a los servicios son referencias de tipos. El adaptador y su contrato se reutilizan del spike. Los métodos del SUT nuevo permanecen pendientes de implementación; el diagrama muestra sus firmas y las dependencias declaradas.

1. Adapter

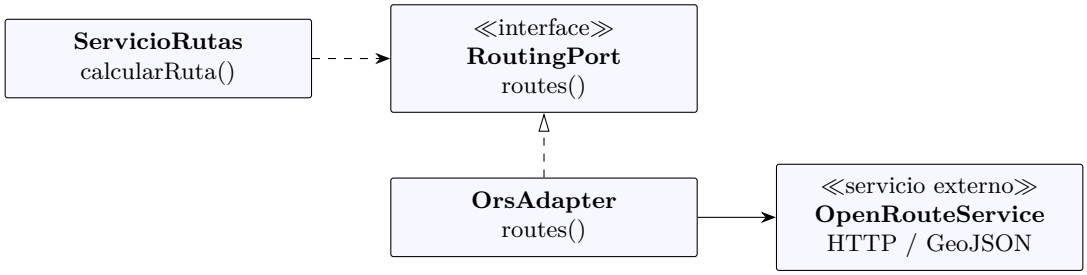
Problema en Vía. OpenRouteService emplea su propio formato de petición y GeoJSON de respuesta. El servicio de rutas necesita trabajar con lugares, geometría, distancia y duración en los tipos de la aplicación.

Aplicación prevista. `ServicioRutas` dependerá de `RoutingPort`. `OrsAdapter` implementará esa interfaz y traducirá la llamada a ORS y su respuesta. El spike ya ofrece esta implementación. La HU-14 la recibe como dependencia en sus pruebas. `OrsAdapter` también adapta geocodificación para otras historias.

UML genérico



UML aplicado a Vía



Ventajas. Aísla los formatos y errores de ORS, facilita probar el servicio y permite incorporar otro proveedor que cumpla la interfaz.

Inconvenientes. Exige mantener las conversiones y revisar qué conceptos son comunes a los proveedores. Un contrato demasiado específico de ORS dificultaría sustituirlo.

Alternativa. Llamar a ORS directamente desde el servicio requiere menos clases, pero mezcla reglas del proyecto con detalles HTTP y de GeoJSON. Adapter encaja por la dependencia externa que ya utiliza Vía.

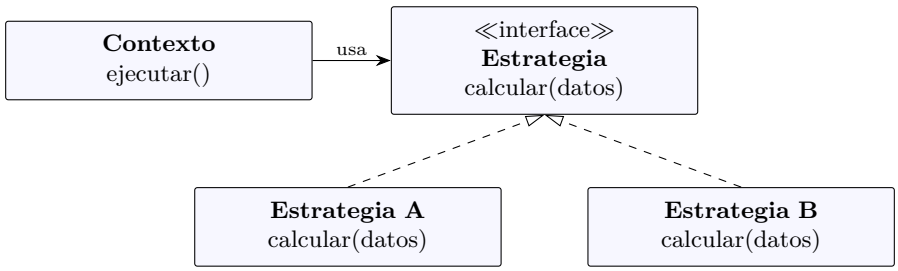
Evidencia. `src/adapters/providers.ts`, clase `OrsAdapter`; contrato en `src/domain/types.ts`. Los diagramas muestran el papel de rutas.

2. Strategy

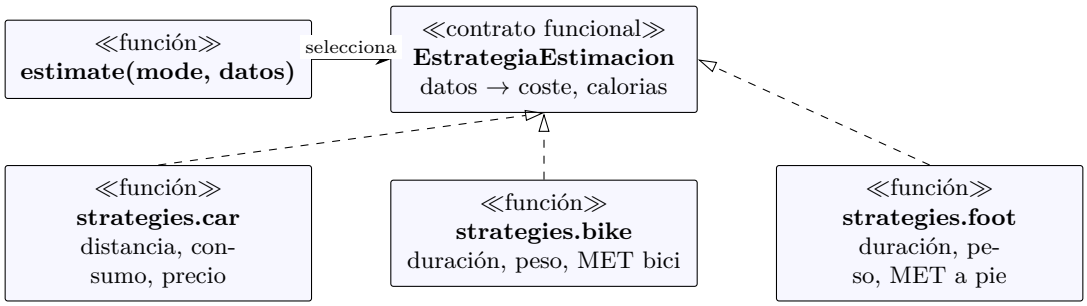
Problema en Vía. Los modos coche, bicicleta y a pie requieren reglas de estimación distintas. El coche combina distancia, consumo y precio energético. Bicicleta y marcha estiman calorías mediante duración, peso y el MET de cada actividad.

Aplicación prevista. La planificación seleccionará una estrategia según el transporte. Las HU-15, HU-16 y HU-17 usarán estos cálculos. En el spike, las estrategias son funciones del objeto `strategies` y `estimate` selecciona una de ellas. En TypeScript se puede expresar este patrón con funciones que compartan la misma firma.

UML genérico



UML aplicado a Vía



Ventajas. Localiza las reglas de cada transporte y permite comprobar los cálculos por separado. La planificación utiliza una operación común.

Inconvenientes. Hay que mantener un contrato coherente y decidir qué datos recibe cada estrategia. Añadir estrategias para diferencias mínimas puede complicar el diseño.

Alternativa. Un bloque condicional dentro del planificador sería suficiente con pocas reglas estables, pero concentraría allí cada nueva fórmula. Combustible y electricidad comparten la fórmula distancia/100 × consumo × precio; por eso se propone seleccionar por modo de transporte, como en el spike.

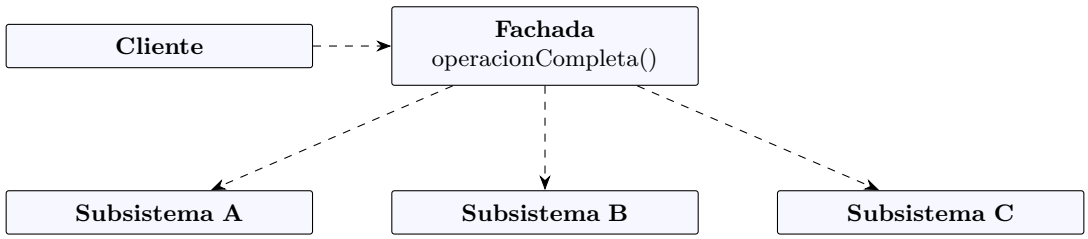
Evidencia. `src/domain/cost.ts`. `EstrategiaEstimacion` es el nombre del contrato común representado en el UML; las funciones del spike lo cumplen de forma estructural.

3. Facade

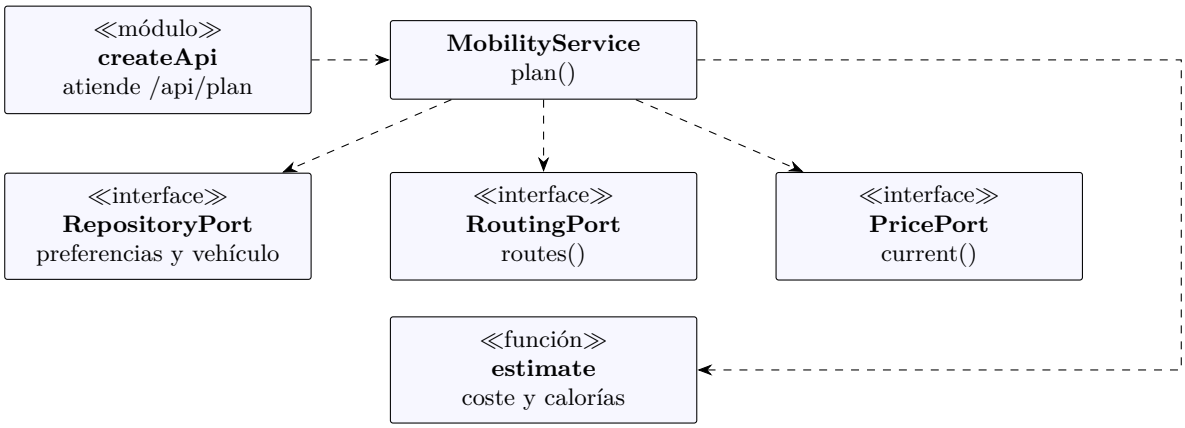
Problema en Vía. Planificar un viaje implica comprobar los extremos, recuperar preferencias y vehículo, consultar precios y rutas, calcular estimaciones y ordenar alternativas. Repetir esa coordinación en cada consumidor aumentaría el acoplamiento.

Aplicación prevista. Una operación de planificación ofrecerá al cliente un resultado conjunto. La HU-14 aporta las rutas; las HU-15 a HU-18 incorporan estimaciones y criterio de selección. En el spike, `MobilityService.plan` ya realiza esa coordinación.

UML genérico



UML aplicado a Vía



Ventajas. Centraliza la secuencia de planificación y ofrece un punto de entrada sencillo a los consumidores. Los componentes especializados conservan sus cálculos y consultas.

Inconvenientes. La fachada puede crecer demasiado si absorbe todas las responsabilidades. El spike combina muchas operaciones en `MobilityService`; en el proyecto se vigilará su tamaño y se delegará el trabajo especializado.

Alternativa. Coordinar desde el manejador HTTP simplifica inicialmente la estructura, pero mezcla protocolo y planificación. Una fachada permite reutilizar la operación desde otros consumidores.

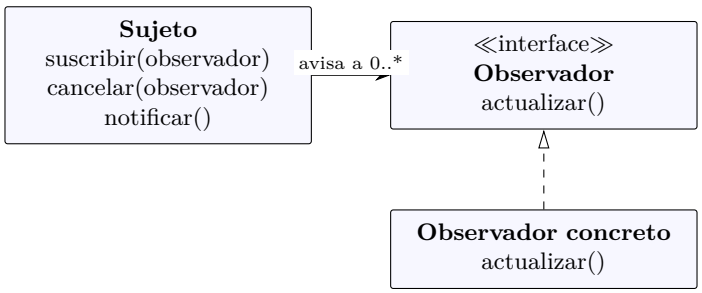
Evidencia. `src/services.ts`, método `plan`; entrada HTTP en `src/http.ts`. Se muestra la parte de planificación del servicio.

4. Observer

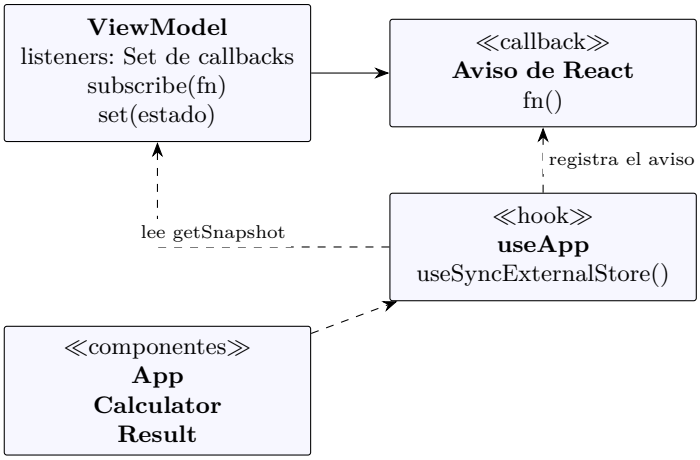
Problema en Vía. Varias vistas consumen el estado compartido del usuario y de la planificación. Al modificarlo, las vistas activas necesitan enterarse para mostrar los datos actuales. El modelo necesita permitir la suscripción y su cancelación.

Aplicación prevista. Se conservará un modelo observable para las vistas. En el spike, `ViewModel.subscribe` guarda callbacks y devuelve una función para cancelar la suscripción. `set` sustituye el estado y ejecuta los callbacks. El hook `useApp` lo enlaza con React.

UML genérico



UML aplicado a Vía



Ventajas. Las vistas se actualizan tras cambios como guardar lugares o recibir rutas. El modelo mantiene callbacks en lugar de referencias a componentes concretos.

Inconvenientes. Las suscripciones deben cancelarse al desmontar las vistas. Un estado muy amplio puede provocar actualizaciones de más componentes de los necesarios.

Alternativa. Mantener todo el estado en un componente padre y pasarlo por propiedades puede servir con una interfaz pequeña. El spike ya comparte un modelo y tres componentes se suscriben mediante el hook.

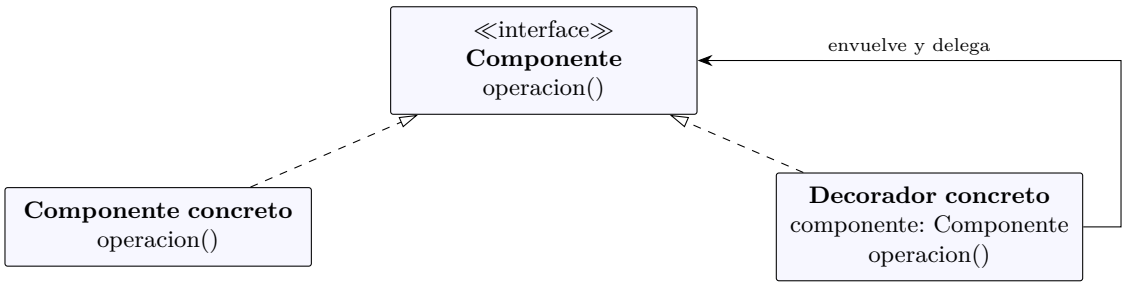
Evidencia. `web/model.ts`, `ViewModel` y `useApp`; consumidores en `web/main.tsx`. Los nodos marcados como hook, callback y componentes representan funciones de TypeScript.

5. Decorator

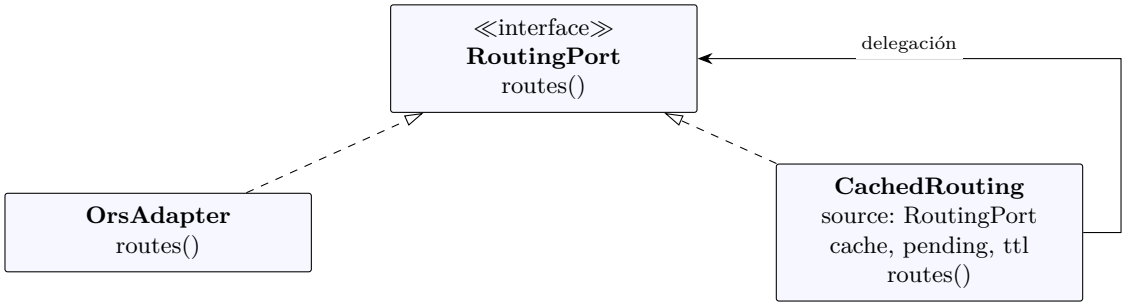
Problema en Vía. Repetir una consulta de ruta con los mismos extremos y modo implica otra llamada externa y espera. Varias solicitudes simultáneas iguales también pueden duplicar la consulta. Se quiere añadir una política de caché al proveedor elegido.

Aplicación prevista. `CachedRouting` envolverá un `RoutingPort` y ofrecerá el mismo método `routes`. En el spike guarda hasta cien resultados recientes durante tres minutos, agrupa solicitudes simultáneas iguales y elimina las entradas caducadas al consultarlas. Es una decisión técnica de optimización que se propone conservar.

UML genérico



UML aplicado a Vía



Colaboración. El servidor crea `new CachedRouting(ors)`. La clave incluye coordenadas de origen y destino y el modo. Si el resultado sigue vigente se devuelve una copia. En otro caso se delega en el proveedor. Los fallos no se conservan en caché.

Ventajas. Reduce consultas repetidas y tiempo de espera manteniendo el contrato del proveedor. La política se configura al envolverlo.

Inconvenientes. Consume memoria y puede devolver un resultado anterior hasta que caduque. La caducidad debe adecuarse al tipo de información consultada.

Alternativa. Incorporar la caché dentro de `OrsAdapter` reduciría el número de clases, pero uniría política de caché y adaptación de ORS. Decorator permite reutilizar la política con otro `RoutingPort`.

Evidencia. `src/adapters/providers.ts`, `CachedRouting`; composición en `src/server.ts`. Existen comprobaciones de caché en `tests/adapters.test.ts`.

Alcance del documento. Análisis preliminar para la Entrega 2, basado en las 26 HU y en el spike. El SUT nuevo de la IT1 sigue en fase roja. La justificación del patrón de interfaz se preparará en su apartado propio.